ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №4

«Професійне ООП та система типів Python»

з курсу «Професійний Python»
Варіант №3 — «Інтернет-магазин»

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла):Куньо І.М.

Дата: 27.10.26

Львів — 2026

1. Мета роботи

Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів (ABC), протоколів (Protocol), dataclass, properties, dunder-методів, принципів SOLID та сучасної статичної системи типізації (type hints, Generic, TypeVar, TypedDict). Побудувати розширювану архітектуру з використанням впровадження залежностей (Dependency Injection) та перевірити кодову базу за допомогою статичного аналізатора типів mypy в суворому режимі (--strict).

2. Індивідуальний варіант і постановка задачі

Варіант №3: «Інтернет-магазин».

Необхідно реалізувати об'єктну модель, яка включає сутності:

- Product — товар інтернет-магазину;
- Category — товарна категорія;
- InventoryItem — облікова складська одиниця товару з контролем залишку;
- Cart — кошик покупця (користувацька контейнерна колекція);
- StoreCatalog — каталог товарів магазину.

Функціональні вимоги варіанта:

1. Додавання товарів у каталог та реєстрація початкових залишків на складі.
2. Облік кількості товару на складі, списання під час оформлення замовлення та валідація нестачі.
3. Розрахунок загальної вартості залишків на складі.
4. Пошук та фільтрація товарів за категорією.
5. Визначення найдорожчого товару в каталозі.
6. Реалізація узагальненого репозиторію (Generic Repository).
7. Реалізація поліморфної системи розрахунку знижок (pricing/discount abstraction).
8. Абстракція платіжного шлюзу та системи сповіщень через структурні протоколи (Protocol).
9. Сервісний шар (StoreService) з реалізацією Dependency Injection та дотриманням принципів SOLID.

3. Структура проєкту

Проєкт структуровано за модульним принципом, де кожен модуль відповідає за окремий архітектурний шар системи:

```
shop_project/
├── report/
│   └── Вересюк_Ірина_Лаб04_V3.pdf
├── .gitignore
```

├── value_objects.py

├── protocols.py

├── repositories.py

├── models.py

├── services.py

└── main.py

4. Теоретичні засади та принципи ООП

Інкапсуляція та Properties: об'єкти самостійно контролюють свій внутрішній стан і не допускають некоректних змін ззовні (наприклад, заборона від'ємної ціни чи залишку). Декоратор @property надає доступ до атрибута через інтерфейс читання з можливістю валідації в @setter.

Value Objects (dataclass(frozen=True)): об'єкти, що не мають власної ідентичності, визначаються виключно значеннями своїх атрибутів і є незмінними (immutable) після створення.

Композиція проти наслідування: перевага надається відношенню *has-a* (кошик містить товари, позиція складу містить товар), а наслідування *is-a* використовується лише там, де формується суворий підтип поліморфної поведінки.

ABC проти Protocol: ABC задає номінальну типізацію (клас зобов'язаний явно успадкувати базовий клас), а Protocol реалізує структурну підтипізацію (*duck typing*), де клас є сумісним з інтерфейсом, якщо містить потрібні методи, без необхідності явного наслідування.

Dunder-методи: дозволяють інтегрувати доменні сутності у вбудовану модель Python (наприклад, __len__, __contains__, __iter__, __add__).

Принципи SOLID:

SRP (Single Responsibility): кожен клас вирішує одну ізольовану задачу.

OCP (Open/Closed): нові види знижок чи платіжних систем додаються без модифікації сервісу.

LSP (Liskov Substitution): будь-яка політика знижок може бути замінена іншою без порушення роботи сервісу.

ISP (Interface Segregation): протоколи платіжного шлюзу та сповіщень розділені й містять лише потрібні методи.

DIP (Dependency Inversion): високорівневий StoreService залежить виключно від абстракцій (PaymentGateway, Notifier, DiscountPolicy), а не від конкретних реалізацій.

5. Опис створених модулів та повні лістинги коду

5.1. value_objects.py

Модуль містить незмінний об'єкт значення Money, створений за допомогою декоратора @dataclass(frozen=True, slots=True). Об'єкт інкапсулює суму та валюту, забороняє від'ємні значення у __post_init__, реалізує магічні методи додавання грошових величин (__add__), множення на коефіцієнт (__mul__) та форматований рядковий вивід (__str__, __repr__).

```
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True, slots=True)

class Money:

  amount: float

  currency: str = "UAH"


  def __post_init__(self) -> None:

    if self.amount < 0:

      raise ValueError("Грошова сума не може бути від'ємною.")


  def __add__(self, other: Money) -> Money:

    if not isinstance(other, Money):

      return NotImplemented

    if self.currency != other.currency:

      raise ValueError(

        f"Невідповідність валют: {self.currency} != {other.currency}"

      )

    return Money(round(self.amount + other.amount, 2), self.currency)


  def __mul__(self, factor: float | int) -> Money:

    if factor < 0:

      raise ValueError("Множник не може бути від'ємним.")

    return Money(round(self.amount * factor, 2), self.currency)


  def __str__(self) -> str:
```

```
        return f"{self.amount:.2f} {self.currency}"

    def __repr__(self) -> str:
        return f"Money(amount={self.amount!r}, currency={self.currency!r})"
```

5.2. protocols.py

Модуль описує абстракції та контракти системи. За допомогою Protocol задано структурні інтерфейси для ідентифікації сутностей (HasId), відправки повідомлень (Notifier) та проведення оплат (PaymentGateway). За допомогою ABC та @abstractmethod описано базовий клас політики ціноутворення та нарахування знижок DiscountPolicy.

```
from abc import ABC, abstractmethod
from typing import Protocol, runtime_checkable
from value_objects import Money


class HasId(Protocol):
    id: int


@runtime_checkable
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...


@runtime_checkable
class PaymentGateway(Protocol):
    def process_payment(self, amount: Money) -> str:
        ...


class DiscountPolicy(ABC):
    """Абстрактний базовий клас для стратегії знижок."""

    @abstractmethod
    def apply(self, total: Money) -> Money:
        ...
```

5.3. repositories.py

Модуль реалізує параметризований узагальнений клас InMemoryRepository[T] з використанням Generic[T] та обмеженої змінної типу TypeVar("T", bound=HasId). Він надає універсальні методи збереження, пошуку та видалення сутностей за їхнім ID. Також описано структуру ProductPayload за допомогою TypedDict для валідації зовнішніх словникових даних, що надходять у систему.

```
from typing import Generic, TypeVar, TypedDict

from protocols import HasId


T = TypeVar("T", bound=HasId)


class InMemoryRepository(Generic[T]):
    """Універсальний репозиторій для сутностей з ідентифікатором."""


    def __init__(self) -> None:
        self._items: dict[int, T] = {}


    def add(self, item: T) -> None:
        self._items[item.id] = item


    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)


    def all(self) -> list[T]:
        return list(self._items.values())


    def remove(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None


class ProductPayload(TypedDict):
    id: int
    name: str
    category_id: int
    category_name: str
```

price: float

currency: str

5.4. models.py

Основний модуль моделі предметної області. Реалізує сутності Category (dataclass), Product (інкапсуляція ціни через @property та валідація в @setter), InventoryItem (облік складського залишку з методами restock та deduct), CartItem (позиція кошика). Також створено два контейнерні класи: Cart, що підтримує перевантаження dunder-методів __len__, __iter__ та __contains__, та StoreCatalog, що реалізує операції пошуку за категорією та пошуку найдорожчого товару

```
from __future__ import annotations

from dataclasses import dataclass, field

from typing import Iterator

from value_objects import Money


@dataclass(frozen=True)

class Category:

  id: int

  name: str

  description: str = ""


class Product:

  def __init__(self, product_id: int, name: str, category: Category, price: Money) -> None:

    self.id = product_id

    self.name = name

    self.category = category

    self._price = price


  @property

  def price(self) -> Money:

    return self._price


  @price.setter

  def price(self, new_price: Money) -> None:

    if new_price.amount <= 0:
```

```
            raise ValueError("Ціна товару повинна бути більшою за нуль.")
        self._price = new_price

    def __repr__(self) -> str:
        return f"Product(id={self.id}, name={self.name!r}, price={self.price})"

    def __str__(self) -> str:
        return f"{self.name} [{self.category.name}] - {self.price}"

@dataclass
class InventoryItem:
    id: int
    product: Product
    _quantity: int = 0

    def __post_init__(self) -> None:
        if self._quantity < 0:
            raise ValueError("Початкова кількість на складі не може бути від'ємною.")

    @property
    def quantity(self) -> int:
        return self._quantity

    def restock(self, amount: int) -> None:
        if amount <= 0:
            raise ValueError("Кількість для поповнення має бути більшою за нуль.")
        self._quantity += amount

    def deduct(self, amount: int) -> None:
        if amount <= 0:
            raise ValueError("Кількість для списання має бути більшою за нуль.")
        if amount > self._quantity:
```

```
            raise ValueError(f"Недостатньо залишку товару '{self.product.name}'. Залишок: {self._quantity}")
        self._quantity -= amount

    @property
    def total_value(self) -> Money:
        return self.product.price * self._quantity


@dataclass
class CartItem:
    product: Product
    quantity: int

    def __post_init__(self) -> None:
        if self.quantity <= 0:
            raise ValueError("Кількість у кошику має бути більшою за нуль.")

    @property
    def subtotal(self) -> Money:
        return self.product.price * self.quantity


@dataclass
class Cart:
    items: list[CartItem] = field(default_factory=list)

    def add(self, product: Product, quantity: int = 1) -> None:
        for item in self.items:
            if item.product.id == product.id:
                item.quantity += quantity
                return
        self.items.append(CartItem(product=product, quantity=quantity))

    def clear(self) -> None:
```

```
        self.items.clear()

    def __len__(self) -> int:
        return sum(item.quantity for item in self.items)

    def __iter__(self) -> Iterator[CartItem]:
        return iter(self.items)

    def __contains__(self, product_id: int) -> bool:
        return any(item.product.id == product_id for item in self.items)

    @property
    def total(self) -> Money:
        if not self.items:
            return Money(0.0)
        currency = self.items[0].product.price.currency
        total_amount = sum(item.subtotal.amount for item in self.items)
        return Money(round(total_amount, 2), currency)

class StoreCatalog:
    """Контейнер каталогу, що інкапсулює роботу з наявними товарами."""

    def __init__(self) -> None:
        self._products: dict[int, Product] = {}

    def add_product(self, product: Product) -> None:
        self._products[product.id] = product

    def get_by_id(self, product_id: int) -> Product | None:
        return self._products.get(product_id)

    def filter_by_category(self, category_name: str) -> list[Product]:
```

```
        return [
            p for p in self._products.values()
            if p.category.name.casefold() == category_name.casefold()
        ]

    def find_most_expensive(self) -> Product | None:
        if not self._products:
            return None
        return max(self._products.values(), key=lambda p: p.price.amount)

    def __iter__(self) -> Iterator[Product]:
        return iter(self._products.values())

    def __len__(self) -> int:
        return len(self._products)
```

5.5. services.py

Модуль прикладного сервісного шару. Містить реалізації політики знижок (NoDiscount, PercentageDiscount), адаптери для зовнішніх систем (ConsoleNotifier, MockCardPaymentGateway), які сумісні з протоколами без явного наслідування завдяки структурній типізації. Клас StoreService використовує Dependency Injection, агрегуючи каталог, складський репозиторій, платіжний шлюз, систему сповіщень та політику знижок, забезпечуючи валідацію залишків, списання та розрахунок загальної вартості запасів.

```
from protocols import DiscountPolicy, Notifier, PaymentGateway
from value_objects import Money
from models import Cart, StoreCatalog, InventoryItem, Product, Category
from repositories import InMemoryRepository, ProductPayload


class NoDiscount(DiscountPolicy):
    def apply(self, total: Money) -> Money:
        return total


class PercentageDiscount(DiscountPolicy):
    def __init__(self, percent: float) -> None:
        if not (0 <= percent <= 100):
```

```
            raise ValueError("Відсоток знижки має бути в діапазоні [0, 100].")
        self.percent = percent

    def apply(self, total: Money) -> Money:
        coefficient = 1.0 - (self.percent / 100.0)
        return Money(round(total.amount * coefficient, 2), total.currency)

class ConsoleNotifier:
    """Адаптер для Notifier без явного наслідування (Structural Typing)."""

    def send(self, recipient: str, message: str) -> None:
        print(f"[Сповіщення -> {recipient}]: {message}")

class MockCardPaymentGateway:
    """Адаптер для PaymentGateway."""

    def process_payment(self, amount: Money) -> str:
        print(f"[Платіжний шлюз]: Списано {amount}")
        return "TX-77491-SUCCESS"

class StoreService:
    """Application Service: оркеструє роботу каталогу, складу та замовлень (DIP/SRP)."""

    def __init__(
        self,
        catalog: StoreCatalog,
        inventory_repo: InMemoryRepository[InventoryItem],
        payment_gateway: PaymentGateway,
        notifier: Notifier,
        discount_policy: DiscountPolicy,
    ) -> None:
        self._catalog = catalog
```

```
    self._inventory = inventory_repo
    self._payment_gateway = payment_gateway
    self._notifier = notifier
    self._discount_policy = discount_policy

  def register_product_from_payload(self, payload: ProductPayload, initial_stock: int) -> Product:
    category = Category(id=payload["category_id"], name=payload["category_name"])
    product = Product(
      product_id=payload["id"],
      name=payload["name"],
      category=category,
      price=Money(payload["price"], payload["currency"]),
    )
    self._catalog.add_product(product)
    item = InventoryItem(id=product.id, product=product, _quantity=initial_stock)
    self._inventory.add(item)
    return product

  def calculate_total_inventory_value(self) -> Money:
    all_items = self._inventory.all()
    if not all_items:
      return Money(0.0)
    total = sum(item.total_value.amount for item in all_items)
    return Money(round(total, 2), all_items[0].total_value.currency)

  def checkout(self, customer_email: str, cart: Cart) -> Money:
    if len(cart) == 0:
      raise ValueError("Кошик порожній.")

    # Перевірка наявності на складі
    for cart_item in cart:
      stock = self._inventory.get(cart_item.product.id)
```

```
        if stock is None or stock.quantity < cart_item.quantity:
            available = stock.quantity if stock else 0
            raise ValueError(
                f"Товар '{cart_item.product.name}' недоступний у кількості {cart_item.quantity} шт. (в наявності: {available})"
            )

    final_price = self._discount_policy.apply(cart.total)
    tx_id = self._payment_gateway.process_payment(final_price)

    # Списання зі складу
    for cart_item in cart:
        stock = self._inventory.get(cart_item.product.id)
        if stock:
            stock.deduct(cart_item.quantity)

    self._notifier.send(
        customer_email,
        f"Замовлення успішно оплачено на суму {final_price}. Код транзакції: {tx_id}",
    )
    cart.clear()
    return final_price
```

5.6. main.py

Виконавча точка входу в програму. Створює об'єкти інфраструктури, конфігурує сервісний шар за допомогою Dependency Injection, завантажує товари з TypedDict, розраховує вартість складу, виконує фільтрацію, демонструє роботу з кошиком (dunder-методи) та проводить успішну оплату замовлення зі списанням залишків.

```
from models import Cart, StoreCatalog, InventoryItem
from repositories import InMemoryRepository, ProductPayload
from services import (
    ConsoleNotifier,
    MockCardPaymentGateway,
    PercentageDiscount,
    StoreService,
```

```
)


def main() -> None:
    # Ініціалізація інфраструктури
    catalog = StoreCatalog()
    inventory_repo = InMemoryRepository[InventoryItem]()
    notifier = ConsoleNotifier()
    payment_gw = MockCardPaymentGateway()
    discount = PercentageDiscount(percent=10.0)  # Знижка 10%

    service = StoreService(
        catalog=catalog,
        inventory_repo=inventory_repo,
        payment_gateway=payment_gw,
        notifier=notifier,
        discount_policy=discount,
    )

    # Додавання товарів через TypedDict
    p1_data: ProductPayload = {
        "id": 1,
        "name": "Ноутбук ASUS ROG",
        "category_id": 101,
        "category_name": "Ноутбуки",
        "price": 45000.0,
        "currency": "UAH",
    }
    p2_data: ProductPayload = {
        "id": 2,
        "name": "Миша Logitech MX Master 3S",
        "category_id": 102,
        "category_name": "Периферія",
```

```
    "price": 4200.0,
    "currency": "UAH",
}
p3_data: ProductPayload = {
    "id": 3,
    "name": "Клавіатура Keychron Q1",
    "category_id": 102,
    "category_name": "Периферія",
    "price": 7800.0,
    "currency": "UAH",
}

laptop = service.register_product_from_payload(p1_data, initial_stock=5)
mouse = service.register_product_from_payload(p2_data, initial_stock=15)
keyboard = service.register_product_from_payload(p3_data, initial_stock=8)

print("--- 1. Облік залишків та оцінка складу ---")
print(f"Загальна вартість усіх запасів на складі: {service.calculate_total_inventory_value()}")

print("\n--- 2. Пошук та фільтрація в каталозі ---")
peripherals = catalog.filter_by_category("Периферія")
print(f"Товари у категорії 'Периферія': {[p.name for p in peripherals]}")

most_expensive = catalog.find_most_expensive()
if most_expensive:
    print(f"Найдорожчий товар: {most_expensive.name} ({most_expensive.price})")

print("\n--- 3. Робота з кошиком (Dunder methods) ---")
cart = Cart()
cart.add(laptop, quantity=1)
cart.add(mouse, quantity=2)
```

```
    print(f"Кількість одиниць товару в кошику (__len__): {len(cart)}")
    print(f"Чи є ноутбук (id=1) в кошику (__contains__): {1 in cart}")
    print(f"Початкова сума кошика: {cart.total}")

    print("\n--- 4. Оформлення замовлення (Checkout) ---")
    final_paid = service.checkout(customer_email="student@univ.edu.ua", cart=cart)
    print(f"Підсумкова сплачена сума зі знижкою: {final_paid}")

    # Перевірка списання зі складу
    laptop_stock = inventory_repo.get(laptop.id)
    print(f"Залишок ноутбуків після покупки: {laptop_stock.quantity if laptop_stock else 0} шт.")

if __name__ == "__main__":
    main()
```

6. Робота з Git

У проєкті виконано три змістовні логічні коміти, які фіксують створення базових контрактів, моделей і бізнес-логіки:

Коміт 1: Базова структура, Value Objects та протоколи

```
Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/shop_project (master)
$ git add .gitignore value_objects.py protocols.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/shop_project (master)
$ git commit -m "Create project structure, value objects and core protocols"
[master (root-commit) f807f40] Create project structure, value objects and core protocols
 3 files changed, 67 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 protocols.py
 create mode 100644 value_objects.py
```

Ініціалізовано репозиторій, додано правила .gitignore для системних кешів. Реалізовано незмінний об'єкт Money та базові інтерфейси (HasId, PaymentGateway, Notifier, DiscountPolicy).

Коміт 2: Моделі домену, колекції та репозиторій

```
Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/shop_project (master)
$ git add models.py repositories.py
```

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/shop_project (master)

$ git commit -m "Implement domain entities, container collections and generic repository"

[master 0e1888e] Implement domain entities, container collections and generic repository

2 files changed, 174 insertions(+)

create mode 100644 models.py

create mode 100644 repositories.py

Зафіксовано доменні класи Product, Category, InventoryItem, контейнери Cart і StoreCatalog з dunder-методами, а також узагальнений InMemoryRepository[T] та схему ProductPayload.

Коміт 3: Сервісний шар, адаптери та точка входу

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/shop_project (master)

$ git add services.py main.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/shop_project (master)

$ git commit -m "Add application store service, dependency injection and demo entry point"

[master 49fe498] Add application store service, dependency injection and demo entry point

2 files changed, 189 insertions(+)

create mode 100644 main.py

create mode 100644 services.py

Створено конкретні адаптери протоколів, сервіс StoreService з реалізацією Dependency Injection та головний сценарій у main.py.

6.1. Результат виконання git status

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/shop_project (main)

$ git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Робоче дерево є повністю чистим, усі зміни синхронізовані з віддаленим репозиторієм на GitHub, а файли кешу компілятора та муру успішно ігноруються.

6.2. Результат виконання git log –oneline

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/shop_project (main)

$ git log --oneline

49fe498 (**HEAD** -> **main**, **origin/main**) Add application store service, dependency injection and demo entry point

0e1888e Implement domain entities, container collections and generic repository

f807f40 Create project structure, value objects and core protocols

Відображає послідовну та лінійну історію розробки із трьох логічних комітів.

7. Результати запуску програми та статичної типізації

7.1. Фактичний вивід запуску програми (python main.py)

PS D:\Робочий стіл\проф. python\shop_project> python main.py

--- 1. Облік залишків та оцінка складу ---

Загальна вартість усіх запасів на складі: 350400.00 UAH

--- 2. Пошук та фільтрація в каталозі ---

Товари у категорії 'Периферія': ['Миша Logitech MX Master 3S', 'Клавіатура Keychron Q1']

Найдорожчий товар: Ноутбук ASUS ROG (45000.00 UAH)

--- 3. Робота з кошиком (Dunder methods) ---

Кількість одиниць товару в кошику (__len__): 3

Чи є ноутбук (id=1) в кошику (__contains__): True

Початкова сума кошика: 53400.00 UAH

--- 4. Оформлення замовлення (Checkout) ---

[Платіжний шлюз]: Списано 48060.00 UAH

[Сповіщення -> student@univ.edu.ua]: Замовлення успішно оплачено на суму 48060.00 UAH. Код транзакції: TX-77491-SUCCESS

Підсумкова сплачена сума зі знижкою: 48060.00 UAH

Залишок ноутбуків після покупки: 4 шт.

PS D:\Робочий стіл\проф. python\shop_project>

Програма успішно виконала всі сценарії: розрахувала вартість запасів на складі, відфільтрувала товари периферії, визначила найдорожчу позицію, перевірила наявність товарів через in, застосувала 10% знижку та провела списання зі складу

7.2. Результат перевірки статичним типізатором mypy –strict

PS D:\Робочий стіл\проф. python\shop_project> mypy --strict .

Success: no issues found in 6 source files

Статичний аналізатор перевірив усі 6 модулів у найсуворішому режимі типізації (--strict) та не виявив жодної помилки сумісності типів.

8. Обов'язковий експеримент із перевіркою типів (Type-checking experiment)

Відповідно до методичних вказівок (розділ 71), для дослідження статичного аналізу коду в проєкт було навмисно внесено три типові помилки статичної типізації: порушення типу повернення методу ([return-value]), невідповідність примітивного типу змінної ([assignment]) та порушення контракту узагальненого репозиторію ([arg-type]). Зафіксовано діагностичні повідомлення муру, після чого вихідний код було виправлено й досягнуто повної валідності (Success: no issues found).

8.1. Опис внесених дефектів та діагностика муру

Помилка 1. Невідповідність типу повернення (Return value mismatch):

Файл: services.py, рядок 12.

Суть помилки: Метод або функція задекларувала повернення значення типу int, але повертає текстовий рядок str.

Повідомлення муру:

services.py:12: error: Incompatible return value type (got "str", expected "int") [return-value]

Статичний аналізатор суворо контролює контракт публічного API функції та забороняє повернення значення несумісного типу.

Помилка 2. Невідповідність типу в операції присвоєння (Assignment mismatch):

Файл: main.py, рядок 13.

Суть помилки: Змінній, для якої явно задано тип int, присвоєно значення типу str.

Повідомлення муру:

main.py:13: error: Incompatible types in assignment (expression has type "str", variable has type "int") [assignment]

Аналізатор фіксує невідповідність примітивних типів даних ще на етапі статичного аналізу до виконання програми в runtime.

Помилка 3. Порушення контракту Generic-типу (Generic bound violation):

Файл: main.py, рядок 19.

Суть помилки: Спроба передати об'єкт класу Category в метод add() екземпляра InMemoryRepository[Product], який параметризований виключно сутністю Product.

Повідомлення муру:

main.py:19: error: Argument 1 to "add" of "InMemoryRepository" has incompatible type "Category"; expected "Product" [arg-type]

Завдяки використанню Generic[T] та TypeVar типізатор гарантує типізаційну безпеку колекції, запобігаючи попаданню сторонніх об'єктів у сховище товарів.

8.2. Фактичне виведення термінала під час виявлення дефектів

PS D:\Робочий стіл\проф. python\shop_project> mypy --strict .

services.py:12: error: Incompatible return value type (got "str", expected "int") [return-value]

main.py:13: error: Incompatible types in assignment (expression has type "str", variable has type "int") [assignment]

main.py:19: error: Argument 1 to "add" of "InMemoryRepository" has incompatible type "Category"; expected "Product" [arg-type]

Found 3 errors in 2 files (checked 6 source files)

8.3. Виправлення коду та фінальний результат статичного аналізу

Після фіксації помилок код було приведено у відповідність до правил типізації:

Приведено до ладу типи повернення функцій у services.py.

Усунуто некоректне присвоєння рядка цілочисельним змінним у main.py.

Забезпечено передачу до репозиторію лише об'єктів узгодженого типу Product.

Повторний запуск mypy в найсуворішому режимі підтвердив відсутність зауважень до типізації всіх 6 модулів проєкту:

PS D:\Робочий стіл\проф. python\shop_project> mypy --strict .

Success: no issues found in 6 source files

Висновок:

Під час виконання лабораторної роботи №4 побудовано типізовану об'єктно-орієнтовану модель інтернет-магазину за принципами SOLID (Варіант №3). Реалізовано сутності Product, Category, InventoryItem, контейнери Cart і StoreCatalog, Value Object Money (frozen dataclass), абстракції ABC і Protocol, dunder-методи, узагальнений репозиторій InMemoryRepository[T] та сервіс із Dependency Injection. Коректність архітектури підтверджено успішним експериментом із навмисними помилками та фінальною перевіркою типізації через mypy --strict (Success: no issues found), а історію проєкту зафіксовано трьома комітами в Git.